

COURSE CODE: CSA1401

SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Compiler Design

COURSE OUTCOME COVERED

CO1: Apply lexical analysis for token specification and recognition using LEX tool. (BL3)

Assessment Tool Weightage: 40%

QUESTIONS: 5

Total Marks:50

Mode: In Class

Hours : 60 mts

Annexure A

ANALYTICAL PROBLEM SOLVING

Code of Conduct:

I 1923 111901 E. Gantham (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: E. Gantham

Q. No	Problem Understanding (20)	Method Selection (20)	Solution Process (25)	Accuracy of Calculation (20)	Interpretation of Results (15)	Total Score (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
OVERALL RUBRICS VALUE						
	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 20	/ 20	/ 25	/ 20	/ 15	/ 100

Annexure A

ANALYTICAL PROBLEM SOLVING

1	Trace the processing of the input expression $P = (a * b) + (b * c) * 2$ through all phases of a compiler. - Identify and list all tokens generated during lexical analysis. - Construct the syntax tree during the parsing phase. - Generate the intermediate code for the given expression. - Apply suitable optimizations to the intermediate code. - Explain the significance of the final target code prod
2	For each of the following Regular Expressions, construct the language. Write any FIVE valid strings for each Regular Expression.
3	i) Find the number of tokens in the following C statement is <pre>main() { int a=10; char b="abc"; in t c = 30; ch ar d = "xyz"; in /* comment */ t m = 40.5; }</pre> ii) Identify the lexical errors in the following C statement is <pre>void main() { int x=10, y=20; char * a; a= &x; x= 1xab; }</pre>
4	Find a regular expression corresponding to each of the following subsets of : - The language of all strings that have an even number of 0's. - The language of all strings that contain at least one 1. - The language of all strings in which both the number of 0's and the number of 1's are odd. - The language of all strings that start with 1 and end with 0. - The language of all strings that do not contain consecutive 1's.
	Consider a lexical analyzer for a simplified programming language. The language consists

5

of the following tokens:

1. Keywords: if, else, while, return, int, float
2. Operators: +, -, *, /, =, ==, <, >
3. Separators: (,), {, }, :, ,
4. Identifiers: A sequence of letters followed by a sequence of digits, e.g., var123
5. Integer Literals: A sequence of digits, e.g., 12345
6. Floating-point Literals: A sequence of digits followed by a dot and another sequence of digits, e.g., 123.45

The lexical analyzer needs to tokenize the following input string:

`int var123 = 12345; float var456 = 123.45; if (var123 == var456) { return; }`

Questions:

- a) How many tokens are there in the given input string?
- b) Provide a list of tokens categorized by their types (keywords, operators, separators, identifiers, integer literals, floating-point literals).
- c) Assuming a finite state machine (FSM) is used for lexical analysis, estimate the number of state transitions required to tokenize the given input string. Provide your reasoning based on the transitions needed for each token type.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/for mula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited unclear or interpretation	No interpretation

1.

Given Expression,

$$p = (a * b) + (b * c) * 2$$

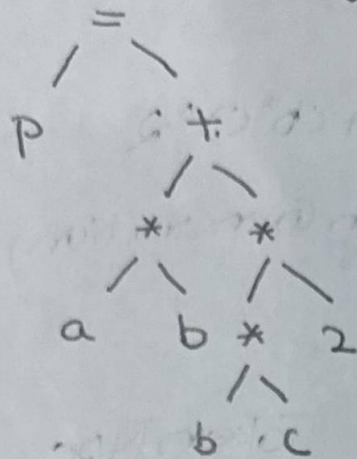
a) Lexical Analysis (Token Generation)

Token List

lexeme	TokenType
p	Identifier
=	Assignment operator
(	left parenthesis
a	Identifier
*	multiplication operator
b	Identifier
)	Right parenthesis
+	Addition operator
(	left parenthesis
b	Identifier
*	multiplication operator
c	Identifier
)	Right parenthesis
*	multiplication operator
2	constant (integer)

Total Tokens = 15

b) parsing phase - Syntax tree



c) Intermediate Code Generation

Three Address code

$$t1 = a * b$$

$$t2 = b * c$$

$$t3 = t2 * 2$$

$$t4 = t1 + t3$$

$$p = t4$$

d) Code optimization

Original TAC

$$t1 = a * b$$

$$t2 = b * c$$

$$t3 = t2 * 2$$

$$t4 = t1 + t3$$

$$p = t4$$

Optimized TAC

$$t1 = a * b$$

$$t2 = b * c$$

$$p = t1 + (t2 * 2)$$

e) Final Target Code

```

LOAD R1, a
MUL R1, b
LOAD R2, b
MUL R2, c
MUL R2, 2
ADD, R1, R2
STORE P, R1

```

2. a) $(aa^*)^*$

Five valid strings

- ϵ (empty string)
- a
- aa
- aaa
- aaaa

b) $(b^*)^*abb$

$(b^*)^* = b^*$ becomes b^*abb

Five valid strings

- abb
- babb
- bbabb
- bbbabb
- bbbbabbb

c) $(a^*b^*)^*aba$ $(a^* + b^*)^*$

Five valid strings

- aba
- aaba

- abab
- baba
- aababb

d) $(cab)^*$

five valid strings

- ϵ (empty string)

- ab

- abab

- ababab

- abababab

e) $(cab)^* + (a^*b^*)$

five valid strings

- ϵ (empty strings)

- aba

- abababa

- aaa

- aaabbb

3) i) no. of tokens in the following C statement is

```
main() {
```

```
    int a=10;
```

```
    char b="abc";
```

```
    int t c=30;
```

```
    char d="xyz"
```

```
    /*comment*/
```

```
    y
```

```
    m=40.5;
```


Reason: Invalid numeric constant. A hexadecimal number should start with 0x eg 0xAB

Valid tokens

void main int char y a 10 20 * 2 =

{ } ; ;

Correct statement

$x = 0xAB;$ or $x = 427;$

4. i) language of all string that have an even no of 0's
- $1^* (01^* 01^*)^*$

Explanation

1^* allows any number of 1's

$(01^* 01^*)^*$ generates pair of 0's

Hence, the total number of 0's is always even

Example

- ϵ
- 11
- 00
- 1001
- 101001

ii) The language of all strings that contain at least one

$(0+1)^* 1 (0+1)^*$: Regular Expression

Ignore spaces and comments

Identify Tokens

No. of tokens

Line	Tokens	No. of tokens
main()	main()	3
{	{	1
int a=10;	int, a, =, 10;	5
char b="abc";	char, b, =, "abc";	5
int c=30;	int, c, =, 30;	6
char d="xyz";	char, d, =, "xyz";	6
/*comment*/ int m=40.5;	int, m, =, 40.5;	6
}	}	1

Total No. of tokens: 33 tokens

ii) Identify lexical Errors

```
void main() {
    int x=10, y=20;
    char *a;
    a = &x;
    x = |xab; }
```

Statement	Error
x = xab	xab

Explanation:

- $(0+1)^*$ allows any combination of 0's and 1's
- 1 guarantee atleast one occurrence of 1
- $(0+1)^*$ allows any remaining symbols.

Example strings

- 1
- 10
- 01
- 101
- 00100

iii) The language of all strings in which both the number of 0's and the number of 1's are odd

Regular Expression

$1^*(01^*01^*)^*01^*(10^*10^*)^*$

Explanation:

odd number of 0's

odd number of 1's

Example string

- 01
- 10
- 010
- 101
- 110001

iv) The language of all string that start with 1 and end with 0.

Regular Expression

10^*0

Explanation

First symbol must be 1

Last symbol must be 0

Any combination of 0's and 1's can appear in between

Example

10

110

100

1010

111110

v) The language of all strings that do not contain consecutive 1's

Regular expression

$(0+10)^*(1+e)$

Explanation

- every 1 is either followed by 0 or appears at the end.
- Therefore, no substring 11 can occur.

5. Given input string

```
int var123 = 12345; float var456 = 123.45; if (var123 == var456)
{ return;
}
```

a) Number of Tokens

Token Identification

No	Token	Type
1	int	Keyword
2	var123	Identifier
3	=	operator
4	12345	Integer Literal
5	,	Separator
6	float	Keyword
7	var456	Identifier
8	=	operator
9	123.45	floating-point Literal
10	,	Separator
11	if	Keyword
12	(	Separator
13	var123	Identifier
14	==	operator
15	var456	Identifier

16	)	Separator
17	{	Separator
18	return	Keyword
19	;	Separator
20	}	Separator

Total number of Tokens = 20

b) Tokens Categorized by Type

1. Keywords

• int • float • if • return

Total = 4

2. Operators

• = • == • >=

Total = 3

3. Separators

• ; • { • } • (•) • , • .

Total = 7

4. Identifiers

• var123 • var456 • var123 • var456

Total = 4

5. Integer Literals

• 12345

Total = 1

6. Floating-point Literals

• 123.45

Total = 1

c) FSM State Transition Estimation

- Start state $\rightarrow$ Identifier/keyword $\rightarrow$ final state
- Start state $\rightarrow$ Number $\rightarrow$ final state
- Start state $\rightarrow$ floating number $\rightarrow$ final state
- Start state $\rightarrow$ operator $\rightarrow$ final state
- Start state $\rightarrow$ Separator $\rightarrow$ final state

Estimated state Transitions

Token Type	No. of Tokens	Approx. Transition Per Token	Total Transitions
Keywords	4	2	8
Identifiers	4	2	8
Operators	3	2	6
Sepriators	7	2	14
Integer Literal	1	2	2
floating literal	1	3	3

Total Estimated FSM state Transitions

$$8 + 8 + 6 + 14 + 2 + 3 = 41$$

Reasoning

- Keywords: Start $\rightarrow$ Keyword $\rightarrow$ Accept (2 transitions)
- Identifiers: Start $\rightarrow$ Identifier $\rightarrow$ Accept (2)
- Operators: Start $\rightarrow$ Operator $\rightarrow$ Accept (2)
- Separators: Start $\rightarrow$ Separator $\rightarrow$ Accept (2)
- Integers: Start $\rightarrow$ Integer $\rightarrow$ Accept (2)
- Floating-point literal: Start $\rightarrow$ Integer $\rightarrow$
Decimal point $\rightarrow$ Fraction $\rightarrow$ Accept (3 transitions)